```
"""
notifier.py — checks upcoming events and
sends "move your car" emails.

Run this on a schedule (e.g. cron every 30
minutes):
    */30 * * * * cd /path/to/spursparking
&& python3 notifier.py

Requires environment variables for SMTP
(see .env.example):
    SMTP_HOST, SMTP_PORT, SMTP_USER,
SMTP_PASSWORD, FROM_EMAIL
"""
import os
import smtplib
from datetime import datetime, timedelta
from email.mime.text import MIMEText

import db
from rules import notification_times,
DEFAULT_RULES

SMTP_HOST = os.environ.get("SMTP_HOST",
"smtp.gmail.com")
SMTP_PORT = int(os.environ.get("SMTP_PORT",
"587"))
SMTP_USER = os.environ.get("SMTP_USER", "")
SMTP_PASSWORD =
os.environ.get("SMTP_PASSWORD", "")
```

```
FROM_EMAIL = os.environ.get("FROM_EMAIL",
SMTP_USER)

# Window either side of "now" in which a
scheduled notification counts as due.
# Should be >= how often this script runs
(e.g. 30 min cron -> 20 min buffer is fine,
# but a slightly wider window guards
against a missed run).
CHECK_WINDOW_MINUTES = 35


def send_email(to_email: str, subject: str,
body: str) -> bool:
    if not SMTP_USER or not SMTP_PASSWORD:
        print(f"[DRY RUN — no SMTP
configured] Would send to {to_email}:
{subject}")
        return True

    msg = MIMEText(body)
    msg["Subject"] = subject
    msg["From"] = FROM_EMAIL
    msg["To"] = to_email

    try:
        with smtplib.SMTP(SMTP_HOST,
SMTP_PORT) as server:
            server.starttls()
            server.login(SMTP_USER,
SMTP_PASSWORD)
            server.sendmail(FROM_EMAIL,
[to_email], msg.as_string())
        return True
```

```
    except Exception as e:
        print(f"Failed to send to
{to_email}: {e}")
        return False


def build_message(notification_type: str,
event: dict, window: dict) -> tuple:
    move_by_str =
window["move_by"].strftime("%A %d %B,
%H:%M")
    event_name = event["event_name"]
    restriction_end_str =
window["restriction_end"].strftime("%H:%M")

    if notification_type == "advance":
        subject = f"Move your car by
{move_by_str} — {event_name} at Tottenham
Hotspur Stadium"
        body = (
            f"Heads up — {event_name} is
happening at Tottenham Hotspur
Stadium.\n\n"
            f"Kick-off / start:
{window['kickoff'].strftime('%A %d %B,
%H:%M')}\n"
            f"Move your car by:
{move_by_str}\n"
            f"Restrictions expected to
clear by: {restriction_end_str}\n\n"
            f"If you don't have a valid
CPZ, Homes for Haringey, or Blue Badge
permit, "
            f"parking in the event day zone
```

```
during restricted hours risks a Penalty "
            f"Charge Notice (currently
£160, reduced to £80 if paid within 14
days).\n\n"
            f"These are planning estimates
based on typical event-day patterns — exact
"
            f"bay suspension times are
confirmed by Haringey Council around 7 days
"
            f"before the event, so double-
check nearer the date.\n\n"
            f"You're receiving this because
you subscribed for event-day alerts."
        )
    else:  # reminder
        subject = f"Reminder: move your car
soon — {event_name} today"
        body = (
            f"Reminder — {event_name}
restrictions are expected to start at
{move_by_str}.\n\n"
            f"If your car is still in the
event day zone without a valid permit, "
            f"move it now to avoid a
Penalty Charge Notice.\n\n"
            f"Restrictions expected to
clear by: {restriction_end_str}"
        )
    return subject, body


def run():
    db.init_db()
```

```
    now = datetime.now()
    window_delta =
timedelta(minutes=CHECK_WINDOW_MINUTES)

    events = db.get_upcoming_events()
    if not events:
        print("No upcoming events.")
        return

    for event in events:
        subscribers =
db.get_active_subscribers()
        for sub in subscribers:
            times = notification_times(
                event["event_date"],
                event["kickoff_time"],

lead_time_hours=sub["lead_time_hours"],

reminder_hours_before=sub["reminder_hours_b
efore"],
                rules=DEFAULT_RULES,
            )

            for notif_type, scheduled_at in
[
                ("advance",
times["advance_notice_at"]),
                ("reminder",
times["reminder_at"]),
            ]:
                already_sent =
db.was_notification_sent(sub["id"],
event["id"], notif_type)
```

```
                due = abs((now -
scheduled_at).total_seconds()) <=
window_delta.total_seconds()

                if due and not
already_sent:
                    subject, body =
build_message(notif_type, event, times)
                    if
send_email(sub["email"], subject, body):

db.mark_notification_sent(sub["id"],
event["id"], notif_type)
                        print(f"Sent
{notif_type} notice to {sub['email']} for
{event['event_name']}")


if __name__ == "__main__":
    run()
```